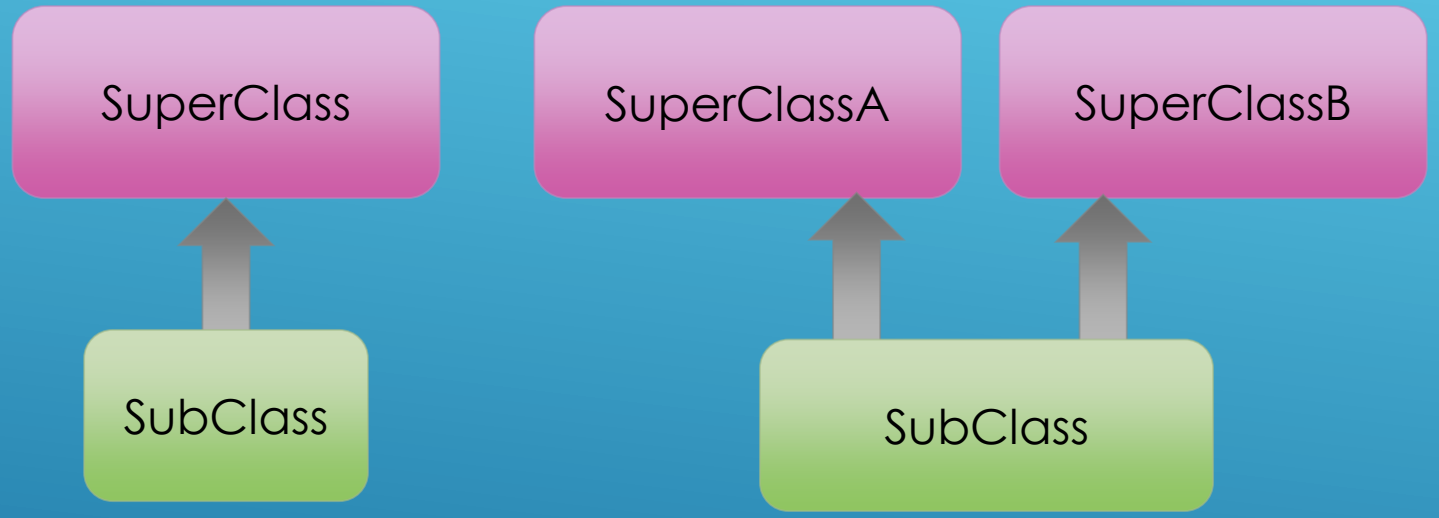
INHERITANCE

Super Class & Sub Class

- We can use each class as a base (**a super class or a foundation**) to define or build another class (**a sub class**).
- It's also possible to use more than one class to define a subclass.



- The left diagram illustrates a "**single inheritance**", and the right one a "multiple inheritance" or "**multi-inheritance**".

```
#include <iostream>
using namespace std;
class Super {
private:
    int storage;
public:
    void put( int val ) {
        storage = val;
    }
    int get( void ) {
        return storage;
    }
};
int main( void ) {
    Super object;
    object.put( 100 );
    object.put( object.get() + 1 );
    cout << object.get() << endl;
    return 0;
}
```

Defining A Simple SuperClass

- We can also write about super classes as base classes, and sub classes as derived classes.
- The class written on the left will serve as a superclass.
- The program will generate the following text as output:

Super Class & Sub Class

- If we want to define a class named Y as a subclass of a superclass named X, we use the following syntax →

  class Y : { visibility specifier } X { . . . };

- The difference from the notation we used before is in the fact that we have to:
  - Place a colon after the subclass class name
  - Optionally place a so-called visibility specifier (we'll return to this soon)
  - Add a superclass name
- If there's more than one superclass, we have to enlist them all using commas as separators, like this:

  class A : X, Y, Z { ... };

Super Class & Sub Class

```
class Sub : Super { };

int main( void ) {
    Sub object;
    object.put(100);
    object.put(object.get() + 1);
    cout << object.get() << endl;
    return 0;
}
```

- We've defined a class named Sub, which is a subclass of a class named Super.
- We may also say that the Sub class is derived from the Super class.
- The Sub class introduces neither new variables nor new functions.
- This doesn't mean that any object of the Sub class inherits all the traits of the Super class, being in fact a copy of the Super class's objects.
- Upon compilation of the following code, we'll get **compilation errors** saying that the put and get methods are inaccessible due to absence of visibility specifier.

when the class is inherited as:	when the component is declared as:	the resulting access inside the subclass is:
public	public	public
	protected	protected
	private	none
protected	public	protected
	protected	protected
	private	none
private	public	private
	protected	private
	private	none

Visibility Specifier

```
#include <iostream>
using namespace std;
class Cat {
public:
    void MakeSound ( void ) {
        cout << "Meow! Meow!" << endl;
    }
};
class Dog {
public:
    void MakeSound ( void ) {
        cout << "Woof! Woof!" << endl;
    }
};
int main ( void ) {
    Cat *a_cat = new Cat();
    Dog *a_dog = new Dog();
    a_cat -> MakeSound();
    a_dog -> MakeSound();
    return 0;
}
```

Type Compatibility

- Each new class constitutes a new type of data.
- Each object constructed on the basis of such a class is like a value of the new type.
- This means that any two objects may (or may not) be compatible in the sense of their types.
- Objects derived from classes which lie in different branches of the inheritance tree are always incompatible.

Overriding Method

- When a subclass declares a method of the name previously known in its superclass, the original method is overridden.
- This means that the subclass hides the previous meaning of the method identifier and any invocation encoded within the subclass will refer to the newer method.
- The code in the next slide demonstrates two important aspects of overriding.
  - Firstly, you can experience the effects of overriding when you invoke the *MakeSound* from within the *a_cat* and *a_dog* objects.
  - Secondly, the effects of the overriding may be reversed.

Overriding Method :: Example

```
#include <iostream>

using namespace std;

class Pet {

protected:

    string Name;

public:

    Pet(string n) { Name = n; }

    void MakeSound(void) {

        cout << Name << "the Pet says: Shh! Shh!"
        << endl; }

};

class Cat : public Pet {

public:

    Cat(string n) : Pet(n) { }

    void MakeSound(void) {

        cout << Name << "the Cat says: Meow!
Meow!" << endl;

    }

};
```

```
class Dog : public Pet {

public:

    Dog(string n) : Pet(n) { }

    void MakeSound(void) { cout << Name << "the Dog
says: Woof! Woof!" << endl; }

};

int main(void) {

    Cat *a_cat;

    Dog *a_dog;

    a_cat = new Cat("Kitty");

    a_dog = new Dog("Doggie");

    a_cat -> MakeSound();

    static_cast<Pet *>(a_cat) -> MakeSound();

    a_dog -> MakeSound();

    static_cast<Pet *>(a_dog) -> MakeSound();

    return 0;

}
```

Polymorphism

- Polymorphism is a method to redefine the behavior of a superclass (but only the one that explicitly agrees to be treated in this way!) without touching its implementation.
- The word "polymorphism" means that the one and same class may show many ("poly" – like in "*polygamy*") forms ("*morphs*") not defined by the class itself, but by its subclasses.
- Polymorphism is the ability to realize class behavior in multiple ways.
- Keyword "virtual" is used in the next code which means that the method won't be overridden within any of the possible subclasses. It also means that the method will be redefined (replaced) at the level of the original class.

Polymorphism :: Example

```
#include <iostream>
using namespace std;
class Pet {
protected:
    string Name;
public:
    Pet(string n) { Name = n; }
    virtual void MakeSound(void) {
        cout << Name << "the Pet says: Shh! Shh!" <<
        endl; }
};
class Cat : public Pet {
public:
    Cat(string n) : Pet(n) { }
    void MakeSound(void) {
        cout << Name << "the Cat says: Meow! Meow!"
<< endl;
    }
};
```

```
class Dog : public Pet {
public:
    Dog(string n) : Pet(n) { }
    void MakeSound(void) { cout << Name << "the Dog
says: Woof! Woof!" << endl; }
};
int main(void) {
    Pet *a_pet1, *a_pet2;
    Cat *a_cat; Dog *a_dog;
    a_pet1 = a_cat = new Cat("Kitty");
    a_pet2 = a_dog = new Dog("Doggie");
    a_pet1 -> MakeSound(); a_cat -> MakeSound();
    static_cast<Pet *>(a_cat) -> MakeSound();
     a_pet2 -> MakeSound(); a_dog -> MakeSound();
    static_cast<Pet *>(a_dog) -> MakeSound();
    return 0;
}
```

- It seems that the Pet class is not always the same – any of its subclasses may change its behavior.
- You cannot predict the behavior of the class containing a virtual function when you don't know all of its subclasses.
- It may also mean that some of the aspects of the class are defined outside the class.
- This is how polymorphism works.

Polymorphism

Passing an Object as a Function Parameter

- Any object may be used as a function parameter and, vice versa, any function may have a parameter as an object of any class.
- All rules regarding parameter passing apply to objects as well.
- The code in the next slide shows two ways to pass an object into a function: by pointer and by reference.
- There are two functions in the example. Both of them have two parameters, but only the second is actually of interest to us. The first function declares a pointer there and the second one a reference. In both cases, the target class is the same: "Pet".

Passing an Object as a Function Parameter

```
#include <iostream>
#include <string>
using namespace std;
class Pet {
protected:
    string name;
public:
    void NameMe(string name) { this -> name = name;
}
    void MakeSound(void) { cout << name << " says:
no comments" << endl; }
};
void PlayWithPetByPointer(string name, Pet *pet) {
    pet -> NameMe(name);
    pet -> MakeSound();
}
```

```
void PlayWithPetByReference(string name, Pet &pet) {
    pet.NameMe(name);
    pet.MakeSound();
}
int main(void) {
    Pet *p1 = new Pet;
    Pet p2;
    PlayWithPetByPointer("anonymous", p1);
    PlayWithPetByReference("no_name", p2);
    PlayWithPetByPointer("no_name", &p2);
    PlayWithPetByReference("anonymous", *p1);
    return 0;
}
```

Passing an Object by Value

```
#include <iostream>
#include <string>
using namespace std;
class Pet {
protected:
    string name;
public:
    void NameMe(string name) { this -> name = name;
}
    void MakeSound(void) { cout << name << " says:
no comments" << endl; }
};
void NamePetByValue(string name, Pet pet) {
    pet.NameMe(name);
}
void NamePetByPointer(string name, Pet *pet) {
    pet -> NameMe(name);
}
```

```
void NamePetByReference(string name, Pet &pet) {
    pet.NameMe(name);
}
int main(void) {
    Pet pet;
    pet.NameMe("no_name");
    NamePetByValue("Alpha", pet);
    pet.MakeSound();
    NamePetByPointer("Beta", &pet);
    pet.MakeSound();
    NamePetByReference("Gamma", pet);
    pet.MakeSound();
    return 0;
}
```

Copying Constructor

- The copying constructors a specific form of constructor designed to make a more or less literal copy of an object.
- You can recognize this constructor by its distinguishable header.
- Assuming that a class is called A, its copying constructor will be declared as:

  A(A &)

- This means that the constructor expects one parameter to be a reference to an object whose content is intended to be copied to the newly created object.

Copying Constructor

- There's no obligation to declare your own copying constructor within any of the classes.
- If there's no explicit copying constructor in some class, an implicit constructor will be used instead.
- The implicit constructor simply clones (bit by bit) the source object, producing a twin copy of it.
- It works satisfactorily in most cases, but requires attention and some consideration when used with objects that contain other objects or pointers.

```
#include <iostream>
using namespace std;
class Class {
     int data;
public:
     Class(int value) : data(value) {}
     void increment(void) { data++; }
     int value(void) { return data; }
};
int main(void) {
     Class o1(123);
     Class o2 = o1;
     Class o3(o2);
     o1.increment();
     cout << o1.value() << endl;
     cout << o2.value() << endl;
     cout << o3.value() << endl;
     return 0;
}
```

Copying Constructors

- The class declared in the example has no explicit copying constructor.
- We expect that an implicit constructor will be used then. It'll be activated twice – during the initializations of the o2 and o3 objects.
- The example reminds us of two possible ways to specify an object's declaration, using the = operator in the classical assignment notation and using the functional notation.

```
#include <iostream>
using namespace std;
class Class {
     int *data;
public:
     Class(int value) {
          data = new int;
          *data = value;
     }
     void increment(void) { (*data)++; }
     int value(void) { return *data; }
};
int main(void) {
     Class o1(123);
     Class o2 = o1;
     Class o3(o2);
     o1.increment();
     cout << o1.value() << endl;
     cout << o2.value() << endl;
     cout << o3.value() << endl;
     return 0;
}
```

Copying Constructors

- The class is slightly modified from the last one.
- Its interface remains the same and the functionality offered by the class is still the same.
- The class's implementation is changed – now the data isn't stored in a regular variable but in a piece of memory allocated by the new operator.
- In effect, the data field is declared as a pointer to the int values (in other words, it's a variable of type int *).

```
#include <iostream>

using namespace std;

class NoConstructorsAtAll {

public:

     int i;

     float f;

     void Display(void) { cout << "i=" << i << ",f="
     << f << endl; }

};

int main(void) {

     NoConstructorsAtAll o1;

     NoConstructorsAtAll *o2;

     o2 = new NoConstructorsAtAll;

     o1.Display();

     o2 -> Display();

     return 0;

}
```

Default Constructors

- The example shows a class that doesn't have a constructor at all.
- The class will be implicitly equipped with the so-called implicit default (parameter-less) constructor but the constructor will do nothing at all.
- The class has no constructor. In effect their fields will not be initialized in any way. The values outputted by the display method are completely random. The number we've seen won't be repeated when you run the program on your computer.

- Inheritance isn't the only form of class/object coexistence. In fact, inheritance is completely useless when we express many of the complicated relationships existing in the real world.
- For example, we can imagine a class designed to gather different kinds of dogs. Making a class and putting it at the bottom of a dog breed inheritance tree isn't a good idea and won't bear desirable fruit.
- The right solution will come to our minds when we understand the actual role of the class – it should gather some dogs so, in fact, it will consists of dogs (like a kennel).

Compositions vs Constructors

Compositions vs Constructors

- This means that the class should include dogs, not inherit them. The kennel has nothing or really very little in common with a dog (for example, there is no kennel that can bark).
- We can say that any complex structure is composed using simpler elements – for example, a car is composed of an engine, transmission, suspension, etc.
- If we imagine all these parts as classes we'll see the car class as a composition that has nothing to do with inheritance.
- As you've probably guessed, building a class is called composition.

```
#include <iostream>
using namespace std;
class A {
public:
    void Do(void) { cout << "A is doing something" << endl; }
};
class B {
public:
    void Do(void) { cout << "B is doing something" << endl; }
};
class Compo {
public:
    A f1; B f2;
};
int main(void) {
    Compo co; co.f1.Do(); co.f2.Do();
    return 0;
}
```

Compositions vs Constructors

- This example shows a simple dummy class consisting (composed) of two objects of two independent classes.
- The program produces the following output:

  A is doing something

  B is doing something

Compositions vs Constructors

```
#include <iostream>

using namespace std;

class A {

public:

    A(A &src) { cout << "copying A..." << endl; }

    A(void) { }

    void Do(void) { cout << "A is doing something"
    << endl; }

};

class B {

public:

    B(B &src) { cout << "copying B..." << endl; }

    B(void){ }

    void Do(void) { cout << "B is doing something"
    << endl; }

};
```

```
class Compo {

public:

    Compo(Compo &src) { cout << "Copying Compo..."
    << endl; }

    Compo(void) { } ;

    A f1;

    B f2;

};

int main(void) {

    Compo co1;

    Compo co2 = co1;

    co2.f1.Do();

    co2.f2.Do();

    return 0;

}
```

- The program produces the following output:

  Copying Compo...

  A is doing something

  B is doing something

- The explicit copying constructor (written by us) has invoked none of the component's copying constructors.
- Unfortunately, defining your own copying constructor means you assume full responsibility for all the activities needed to carry out responsible copying. This means that we need to modify the constructor.

Compositions vs Constructors

Compositions vs Constructors

- One way to do this is to add a line like this one:

```
Compo(Compo &src) : f1(src.f1), f2(src.f2) {
    cout << "Copying Compo..." << endl;
}
```

Instead of

```
Compo(Compo &src) {
    cout << "Copying Compo..." << endl;
}
```

- The solution is correct despite how it looks.

Thank You